REST APIS - AUTHENTICATION AND AUTHORIZATION

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
  - Know the difference between authentication and authorization.
  - Apply different authentication and authorization mechanisms for REST APIs as appropriate to given scenarios.

CONTENTS

- Authentication vs. Authorization
- Importance of Authentication and Authorization
- Some methods for Authentication and Authorization in REST APIs
  - HTTP Basic Authentication
  - API Keys
  - JSON Web Tokens (JWT)
  - OAuth 2.0
    - OpenID Connect (OIDC)
- Summary

AUTHENTICATION VS. AUTHORIZATION

- Should everyone have free access to your resources?
  - No!
  - Only known users should.

- How do we identify if a user is a known user?
  - **Authentication!**
  - [YouTube] Session vs Token Authentication

AUTHENTICATION VS. AUTHORIZATION

- Should all known users be allowed to access all resources?
  - No!
  - They should only have access to what we explicitly want them to access.
- How do we control what they can access?
  - **Authorization!**

AUTHENTICATION VS. AUTHORIZATION

Authorization

What you can do

Authentication

Who you are

Source: https://blog.restcase.com/4-most-used-rest-api-authentication-methods/

AUTHENTICATION VS. AUTHORIZATION

- **Authentication** is when an entity proves an **identity**.
  - Authentication proves that you are who you say you are.

- **Authorization** is when an entity proves a **right to access.**
  - Authorization proves you have the right to make a request.

- In summary:
  - Authentication: Refers to proving correct identity.
  - Authorization: Refers to allowing a certain action.

AUTHENTICATION VS. AUTHORIZATION

Authentication	Authorization
Determines whether users are who they claim to be	Determines what users can and cannot access
Challenges the user to validate credentials (for example, through passwords, answers to security questions, or facial recognition)	Verifies whether access is allowed through policies and rules
Usually done before authorization	Usually done after successful authentication
Generally, transmits info through an ID Token	Generally, transmits info through an Access Token
Generally governed by the OpenID Connect (OIDC) protocol	Generally governed by the OAuth 2.0 framework
Example: Employees in a company are required to authenticate through the network before accessing their company email	Example: After an employee successfully authenticates, the system determines what information the employees are allowed to access

Source: https://auth0.com/docs/get-started/identity-fundamentals/authentication-and-authorization

SOME METHODS FOR AUTH IN REST APIS

- HTTP Basic Authentication
- API Keys
- JSON Web Tokens (JWT)
- OAuth 2.0
  - OpenID Connect (OIDC)

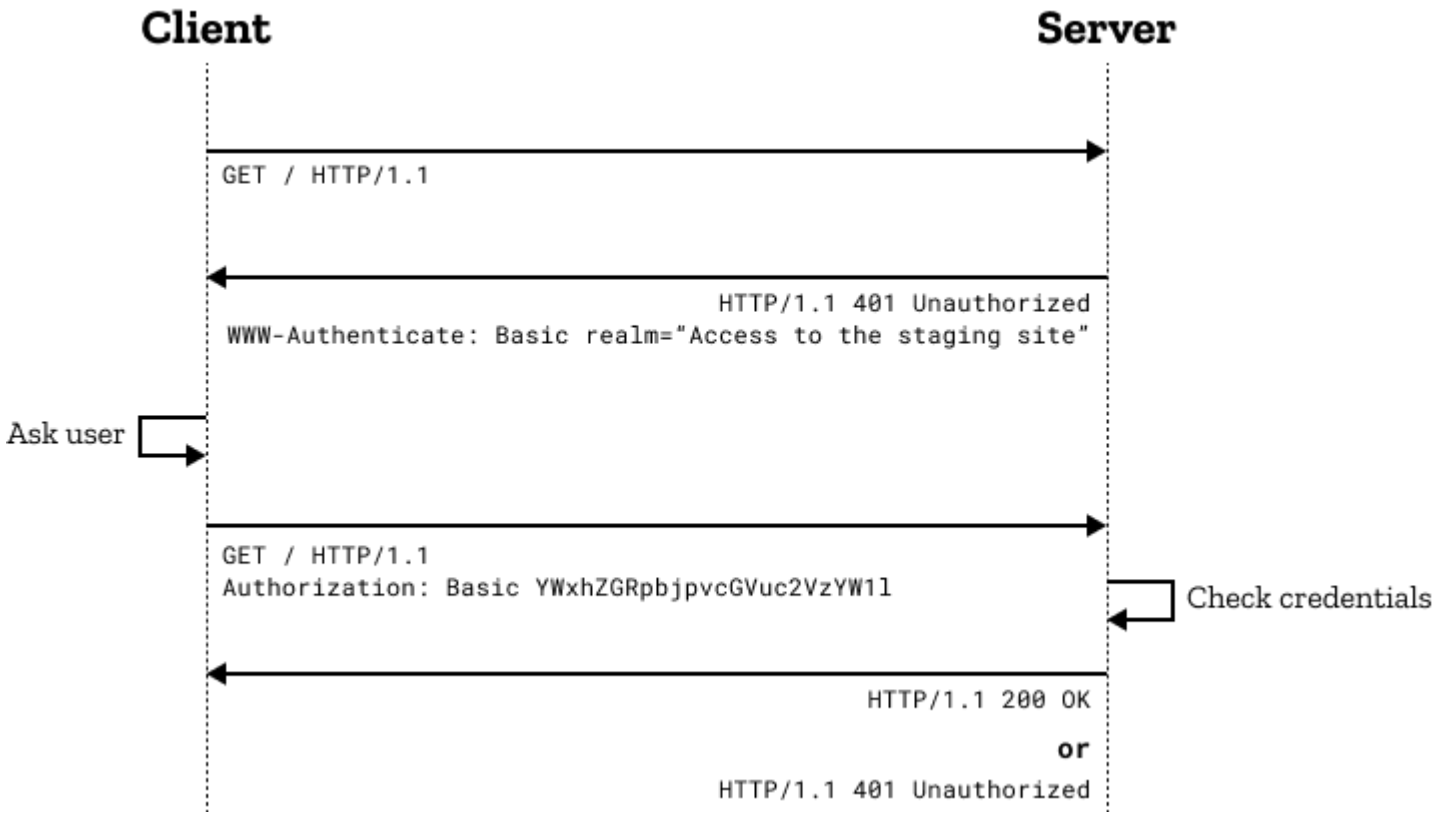
HTTP BASIC AUTHENTICATION

- Most basic form of authentication in APIs.
- The client sends the username and the password of the user encoded in Base64 encoding technique to the server.
- The credentials are sent in the HTTP header with every request.
  - *Authorization: Basic <username:password>*

HTTP BASIC AUTHENTICATION

- Very simple to use as it does not require cookies, session identifiers, login pages.
- Lightweight.
  - Good for use-cases like IoT.
- Rarely recommended due to its inherent security vulnerabilities.
  - For better security use with HTTPS over SSL/ TLS.

HTTP BASIC AUTHENTICATION



Source: https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication

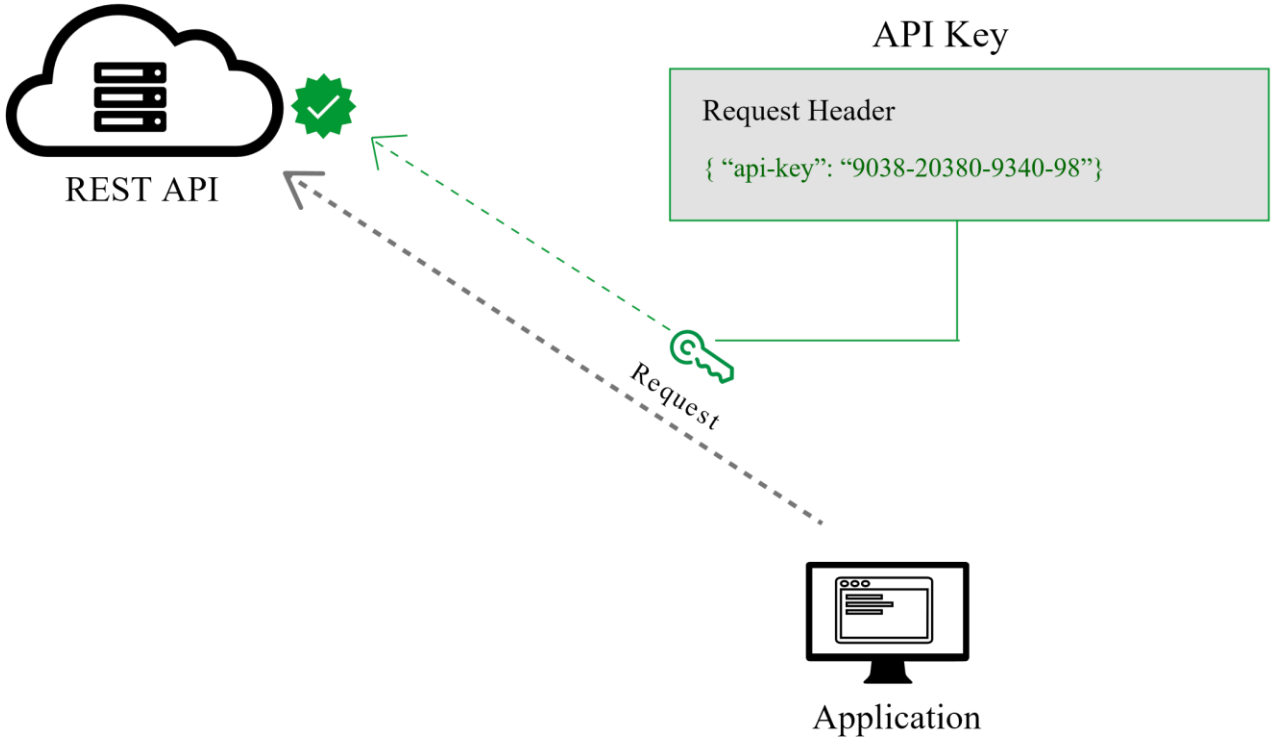
API KEYS

- A **unique generated key value** is assigned for each first-time user.
- Thereafter, the user sends all requests with the key as,
  - a query parameter or
  - in the HTTP header (the standard way)
- Simple, fast and convenient.

API KEYS

- A method of authentication, not authorization.
- No expiration.
  - Once the key is stolen, it may be used indefinitely unless revoked.
- As with Basic HTTP authentication, not recommended to be used as is.
  - For better security use with HTTPS over SSL/ TLS.

API KEYS

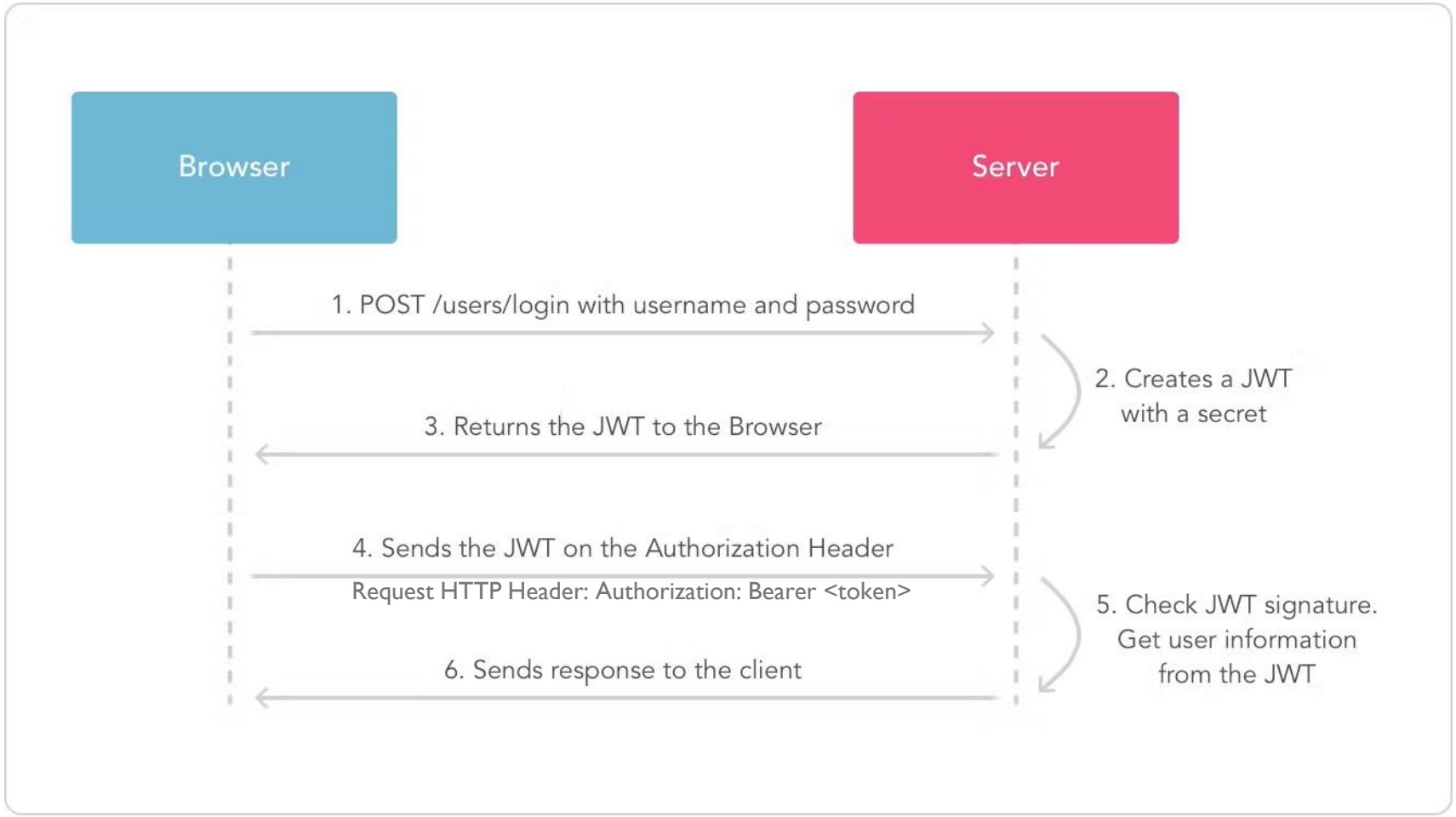


Source: https://blog.restcase.com/4-most-used-rest-api-authentication-methods/

JSON WEB TOKENS (JWT)

- A compact and a self-contained means to securely exchange information between parties as a JSON object.
- Digitally signed, therefore can be verified and trusted.
  - Using secrets
  - Using public/ private key pairs
- Used for user **authorization** after authentication.
- Very popular!

JSON WEB TOKENS (JWT)



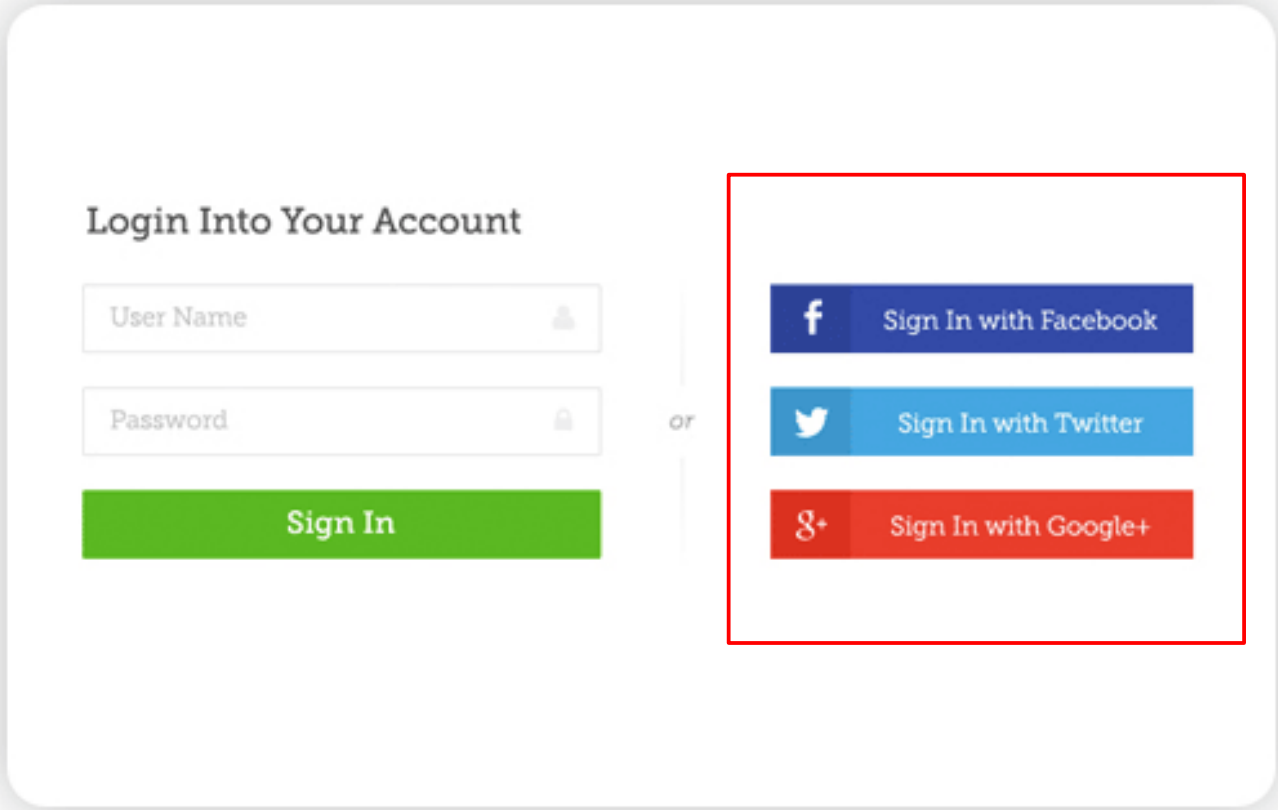
Source: https://lo-victoria.com/introduction-to-rest-api-authentication-methods

JSON WEB TOKENS (JWT)

- JWT Decoder – example JWT demo
- JWT guarantees that data is not tampered because of the signature.
- The contents of a JWT can be seen by anyone that intercepts the token.
  - It is just serialized, not **encrypted**.
  - It is recommended to **use HTTPS** to avoid this problem.
- JWT is pronounced as "Jot".
- JWT is good for implementing stateless APIs.

OAUTH 2.0

Have you seen this before?

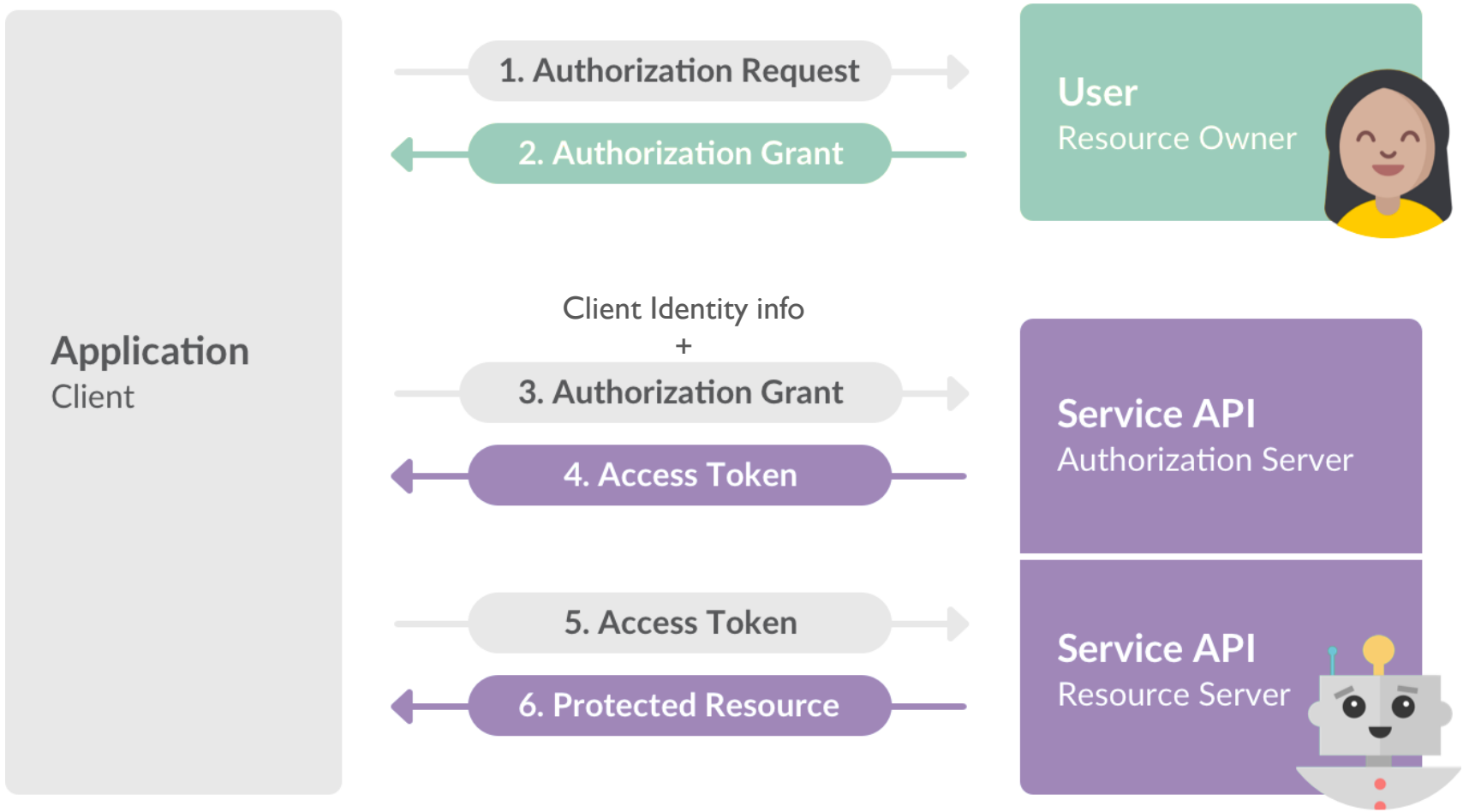


Source: https://lo-victoria.com/introduction-to-rest-api-authentication-methods

OAUTH 2.0

- It is an authorization framework/ protocol.
- OAuth addresses a very specific problem.
  - How can the **resource owner** give a **client**, **scoped access** to a **protected resource**?
- If an implementation also conforms to OpenID Connect specification, it will also support authentication.
  - This is how social authentication works!
- OAuth works by,
  - **Delegating user authentication** to a service that hosts a user account
  - **Authorizing third-party applications** to access that user account

OAUTH 2.0



Source: https://lo-victoria.com/introduction-to-rest-api-authentication-methods

OAUTH 2.0

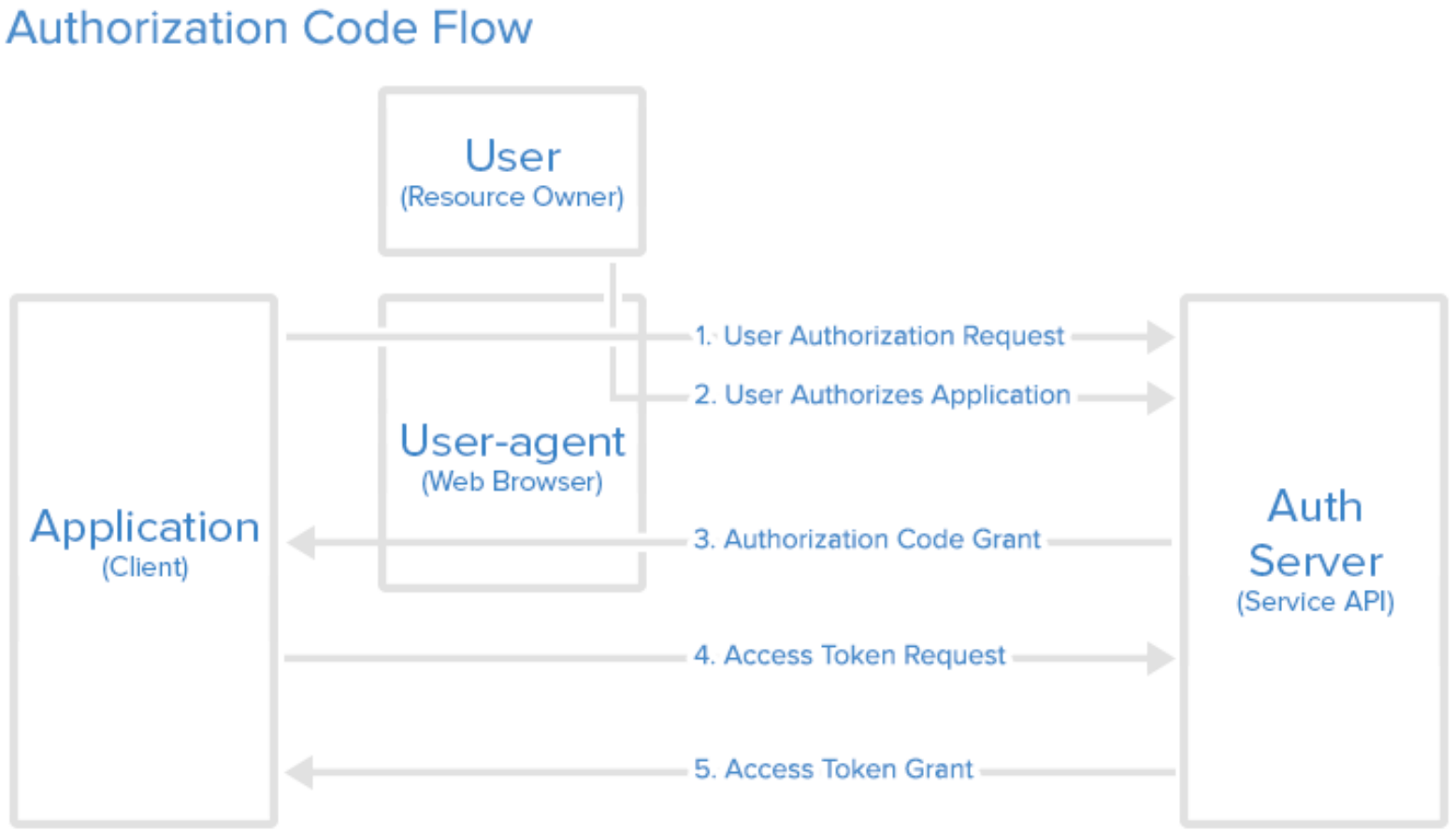
- OAuth Roles
  - **Resource Owner:** The resource owner is the user who authorizes an application to access their account.
  - **Client:** The client is the application that wants to access the user's account.
  - **Authorization Server:** The authorization server verifies the identity of the user then issues access tokens to the client application.
  - **Resource Server:** The resource server hosts the protected resources.

OAUTH 2.0

- An OAuth 2.0 flow (grant type) is a way of retrieving an Access Token.
- OAuth 2.0 supports different authorization flows (also known as grant types).
  - Authorization Code Flow – Widely used for server-side and mobile web applications.
  - PKCE
  - Client Credentials flow etc.

OAUTH 2.0

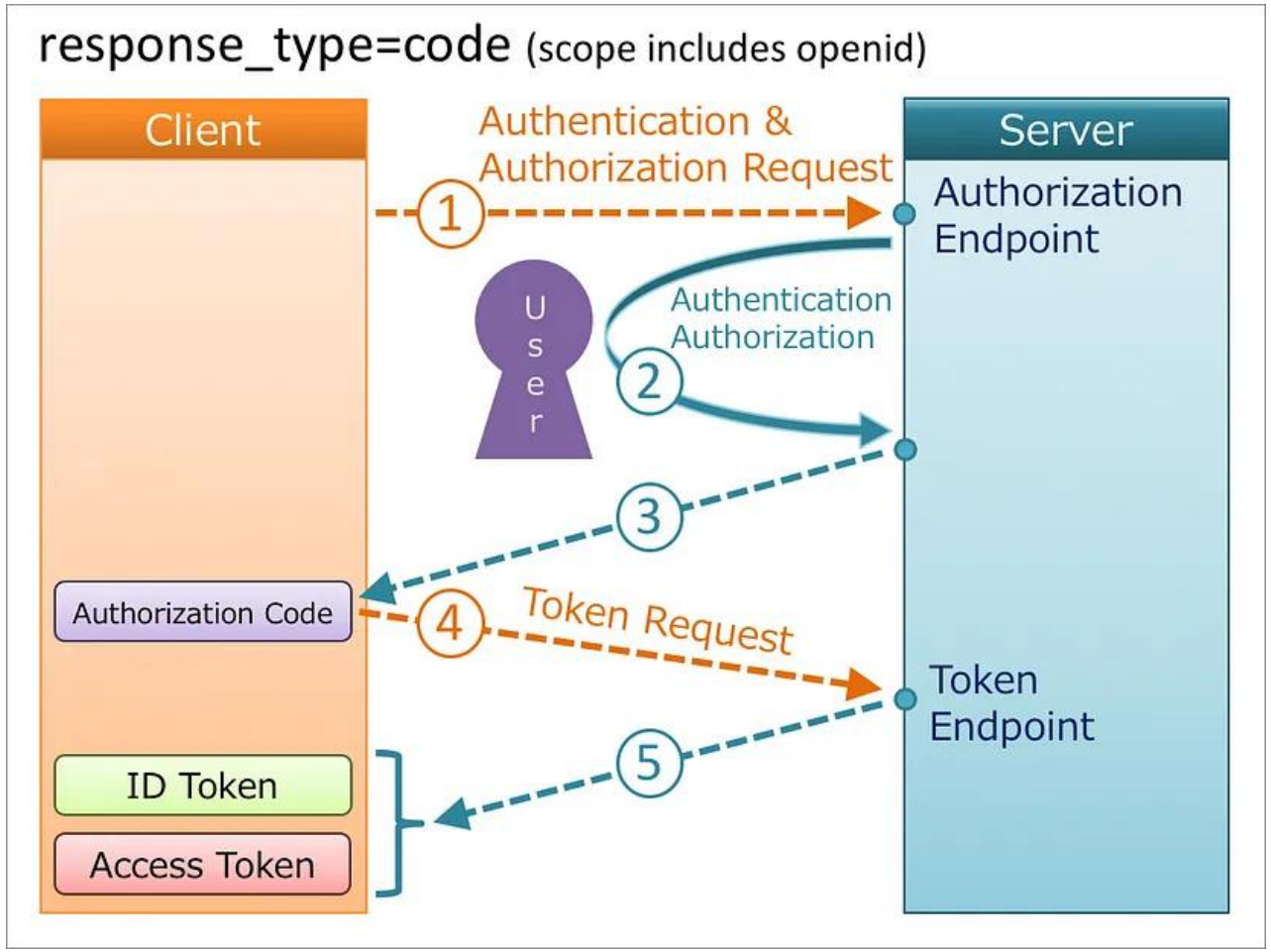
Authorization Code Flow



Source: https://www.digitalocean.com/community/tutorials/an-introduction-to-oauth-2

OPENID CONNECT

Authorization Code Flow
+ with Authentication



Source: https://darutk.medium.com/diagrams-of-all-the-openid-connect-flows-6968e3990660

OAUTH 2.0

- Why use OAuth 2.0?
  - It enables an application to obtain limited access (scopes) to a user's data without the user having to share their password.
- Why use OpenID Connect (which is an extension of OAuth 2.0)?
  - Accelerate Sign Up Process at Your Favorite Websites
  - No need to maintain multiple usernames/ passwords.
  - Minimize password security risks.

OAUTH 2.0

- [YouTube] OAuth 2.0: An Overview

SELF-STUDY ACTIVITY

- There is much more in OAuth 2.0.
  - Read this introduction to OAuth 2.0 to get a more detailed understanding.
- Spring Boot and OAuth 2.0
  - Check out these official tutorials from Spring.

SUMMARY

- Authentication vs. Authorization
- Importance of Authentication and Authorization
- Some methods for Authentication and Authorization in REST APIs
  - HTTP Basic Authentication
  - API Keys
  - JSON Web Tokens (JWT)
  - OAuth 2.0
    - OpenID Connect (OIDC)

REFERENCES

1. 3 Common Methods of API Authentication Explained
2. 4 Most Used REST API Authentication Methods
3. How does HTTPS actually work?
4. HTTP authentication
5. API Keys
6. Introduction to JSON Web Tokens
7. Authorization Code Flow

THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK